

```
In [1]: # Import the Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: # Import the dataset
data = pd.read_csv('311_Service_Requests_from_2010_to_Present.csv')
```

C:\Users\Vishal\AppData\Local\Temp\ipykernel_9184\682472732.py:2: DtypeWarning: Columns (48,49) have mixed types. Specify dtype option on import or set low_memory=False.

```
data = pd.read_csv('311_Service_Requests_from_2010_to_Present.csv')
```

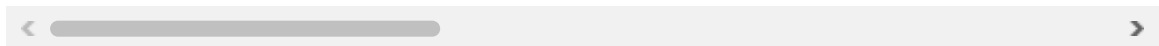
Visualize the dataset

```
In [3]: data.head()
```

Out[3]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk

5 rows × 53 columns

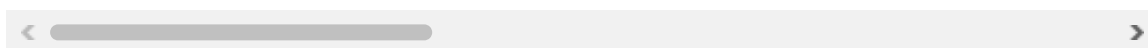


```
In [4]: data.tail()
```

Out[4]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descri
364553	29609918	01/01/2015 12:04:44 AM	01/01/2015 10:22:31 AM	NYPD	New York City Police Department	Illegal Parking	Bloc Hyd
364554	29608392	01/01/2015 12:04:28 AM	01/01/2015 02:25:02 AM	NYPD	New York City Police Department	Noise - Vehicle	Car/T t
364555	29607589	01/01/2015 12:01:30 AM	01/01/2015 12:20:33 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	L Music/F
364556	29610889	01/01/2015 12:01:29 AM	01/01/2015 02:42:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac
364557	29611816	01/01/2015 12:00:50 AM	01/01/2015 02:47:50 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac

5 rows × 53 columns



Print the columns of the DataFrame

In [5]: `data.columns`

```
Out[5]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
               'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
               'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
               'Intersection Street 1', 'Intersection Street 2', 'Address Type',
               'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
               'Resolution Description', 'Resolution Action Updated Date',
               'Community Board', 'Borough', 'X Coordinate (State Plane)',
               'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
               'School Name', 'School Number', 'School Region', 'School Code',
               'School Phone Number', 'School Address', 'School City', 'School State',
               'School Zip', 'School Not Found', 'School or Citywide Complaint',
               'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
               'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
               'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
               'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
              dtype='object')
```

Identify the shape of the dataset

In [6]: `data.shape`

Out[6]: (364558, 53)

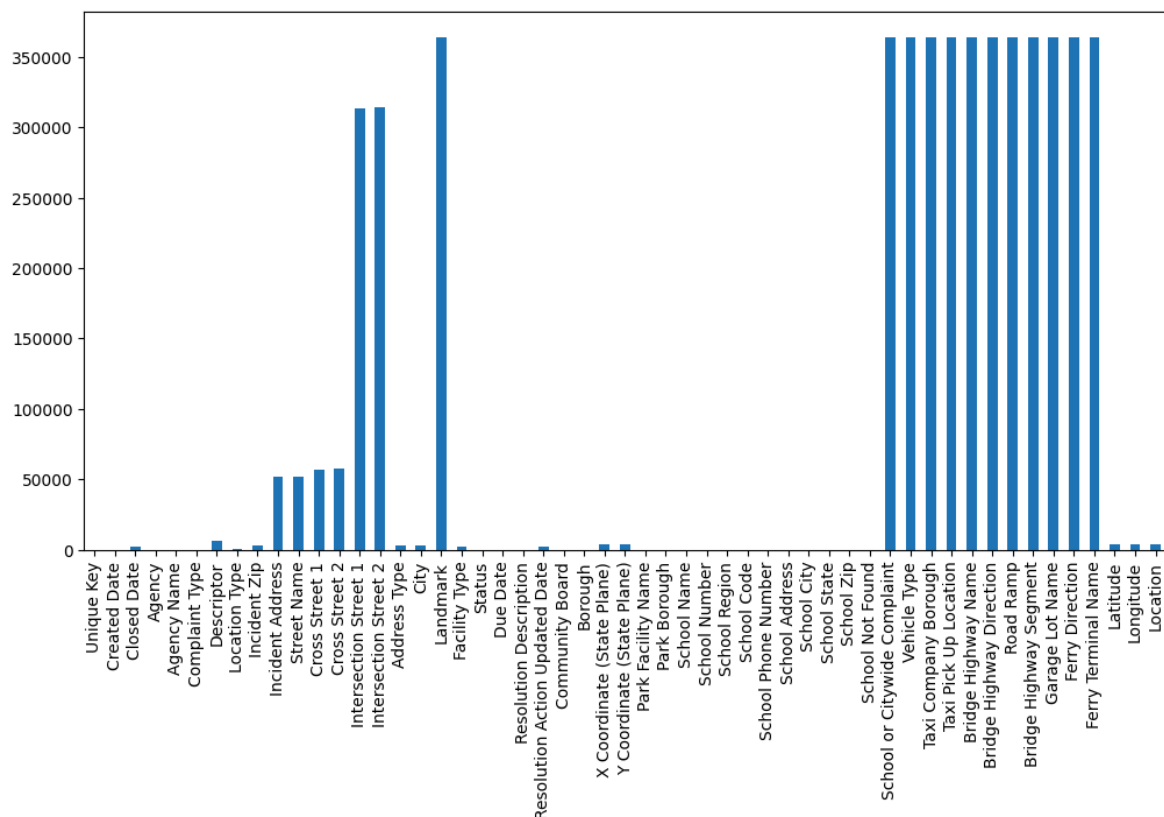
Identify the variables with null values

In [7]: `data.isnull().sum()`

```
Out[7]: Unique Key          0
Created Date              0
Closed Date              2381
Agency                  0
Agency Name             0
Complaint Type           0
Descriptor               6501
Location Type            133
Incident Zip             2998
Incident Address         51699
Street Name              51699
Cross Street 1           57188
Cross Street 2           57805
Intersection Street 1    313438
Intersection Street 2    314046
Address Type             3252
City                     2997
Landmark                 364183
Facility Type            2389
Status                   0
Due Date                  3
Resolution Description    0
Resolution Action Updated Date 2402
Community Board          0
Borough                  0
X Coordinate (State Plane) 4030
Y Coordinate (State Plane) 4030
Park Facility Name       0
Park Borough             0
School Name              0
School Number            0
School Region            1
School Code              1
School Phone Number      0
School Address           0
School City              0
School State             0
School Zip               1
School Not Found         0
School or Citywide Complaint 364558
Vehicle Type             364558
Taxi Company Borough     364558
Taxi Pick Up Location    364558
Bridge Highway Name      364261
Bridge Highway Direction 364261
Road Ramp                364296
Bridge Highway Segment   364296
Garage Lot Name          364558
Ferry Direction          364557
Ferry Terminal Name      364556
Latitude                 4030
Longitude                4030
Location                 4030
dtype: int64
```

Frequency plot of null values

```
In [8]: null_counts = data.isna().sum()
null_counts.plot(kind='bar', figsize=(12,6))
plt.show()
```



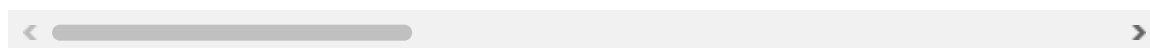
```
In [9]: data = data.drop('Ferry Terminal Name', axis=1)
data = data.drop('Ferry Direction', axis=1)
data = data.drop('Garage Lot Name', axis=1)
data = data.drop('Bridge Highway Segment', axis=1)
data = data.drop('Road Ramp', axis=1)
data = data.drop('Bridge Highway Direction', axis=1)
data = data.drop('Bridge Highway Name', axis=1)
data = data.drop('Taxi Pick Up Location', axis=1)
data = data.drop('Vehicle Type', axis=1)
data = data.drop('School or Citywide Complaint', axis=1)
data = data.drop('Landmark', axis=1)
data = data.drop('Intersection Street 1', axis=1)
data = data.drop('Intersection Street 2', axis=1)
data = data.drop('Taxi Company Borough', axis=1)
```

```
In [10]: data.dropna(axis = 0, inplace = True)
data
```

Out[10]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descri
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Music/f
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Comme Overrr Par
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blor Side
...	...	...	...	...	...	...	...
364553	29609918	01/01/2015 12:04:44 AM	01/01/2015 10:22:31 AM	NYPD	New York City Police Department	Illegal Parking	Blor Hyc
364554	29608392	01/01/2015 12:04:28 AM	01/01/2015 02:25:02 AM	NYPD	New York City Police Department	Noise - Vehicle	Car/T f
364555	29607589	01/01/2015 12:01:30 AM	01/01/2015 12:20:33 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Music/f
364556	29610889	01/01/2015 12:01:29 AM	01/01/2015 02:42:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac
364557	29611816	01/01/2015 12:00:50 AM	01/01/2015 02:47:50 AM	NYPD	New York City Police Department	Blocked Driveway	No Ac

300932 rows × 39 columns



Convert to datetime

```
In [11]: data['Created Date']=pd.to_datetime(data['Created Date'])
```

C:\Users\Vishal\AppData\Local\Temp\ipykernel_9184\3440188760.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
data['Created Date']=pd.to_datetime(data['Created Date'])
```

```
In [12]: data['Closed Date'] = pd.to_datetime(data['Closed Date'])
```

C:\Users\Vishal\AppData\Local\Temp\ipykernel_9184\2084798470.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
data['Closed Date'] = pd.to_datetime(data['Closed Date'])
```

Remove incorrect timeline

```
In [13]: data = data[data['Closed Date'] >= data['Created Date']]
```

```
In [14]: data.isnull().sum()
```

```
Out[14]: Unique Key          0
Created Date              0
Closed Date              0
Agency                  0
Agency Name             0
Complaint Type          0
Descriptor              0
Location Type           0
Incident Zip            0
Incident Address        0
Street Name            0
Cross Street 1          0
Cross Street 2          0
Address Type            0
City                   0
Facility Type           0
Status                 0
Due Date               0
Resolution Description   0
Resolution Action Updated Date 0
Community Board         0
Borough                0
X Coordinate (State Plane) 0
Y Coordinate (State Plane) 0
Park Facility Name      0
Park Borough           0
School Name            0
School Number          0
School Region          0
School Code            0
School Phone Number    0
School Address         0
School City            0
School State           0
School Zip             0
School Not Found       0
Latitude              0
Longitude             0
Location              0
dtype: int64
```

Calculate time elapsed in closed and creation date

```
In [15]: data['Request_Closing_Time'] = data['Closed Date'] - data['Created Date']
```

```
In [16]: data['Request_Closing_Time'].head()
```

```
Out[16]: 0    0 days 00:55:30
1    0 days 01:27:13
2    0 days 04:51:34
3    0 days 07:45:27
4    0 days 03:27:44
Name: Request_Closing_Time, dtype: timedelta64[ns]
```

Convert the calculated date to seconds to get a better representation

```
In [17]: # data['Time_in_seconds'] = data['Time_in_seconds'].dt.total_seconds()
data['Time_in_seconds'] = data['Request_Closing_Time'].dt.total_seconds()
```

```
In [18]: print(data['Time_in_seconds'].describe())
```

```
count    3.009320e+05
mean     1.520342e+04
std       2.063479e+04
min       6.100000e+01
25%      4.670000e+03
50%      9.756000e+03
75%     1.903100e+04
max     2.113926e+06
Name: Time_in_seconds, dtype: float64
```

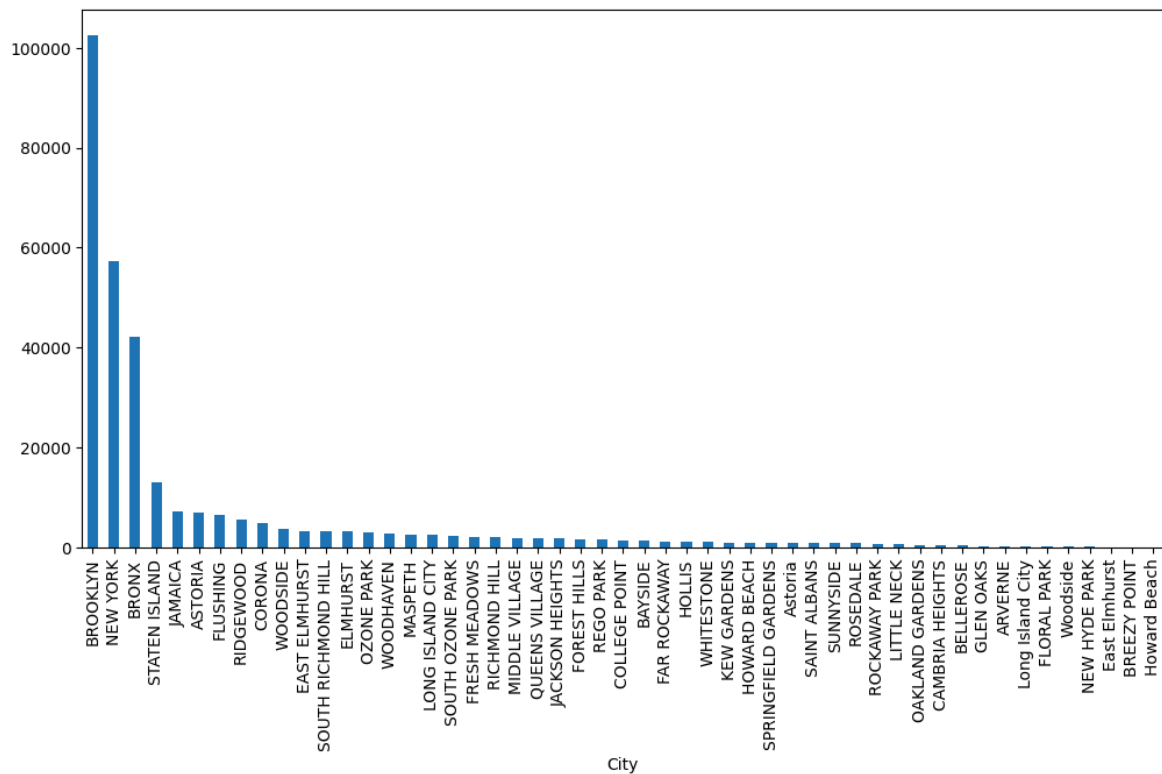
Check the number of null values in the Complaint_Type and City columns

```
In [19]: data[['Complaint_Type', 'City']].isnull().sum()
```

```
Out[19]: Complaint_Type    0
City                      0
dtype: int64
```

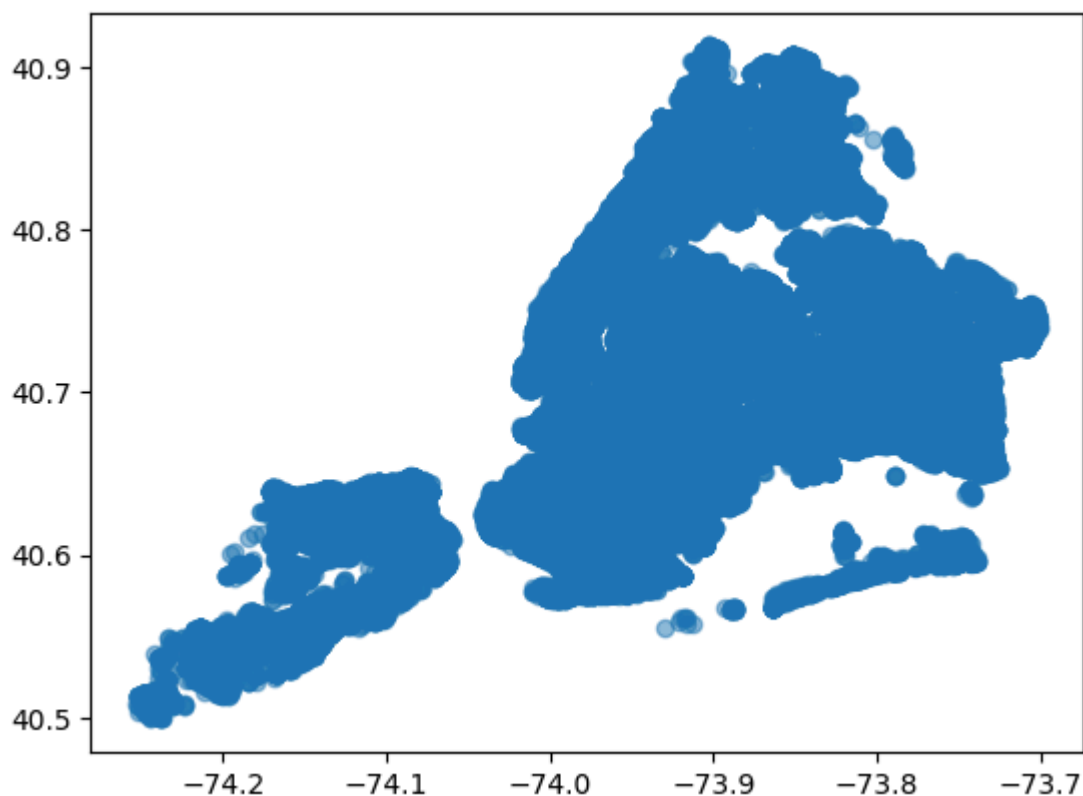
Draw a frequency plot for the complaints in each city

```
In [20]: data['City'].value_counts().plot(kind='bar', figsize=(12,6))
plt.show()
```

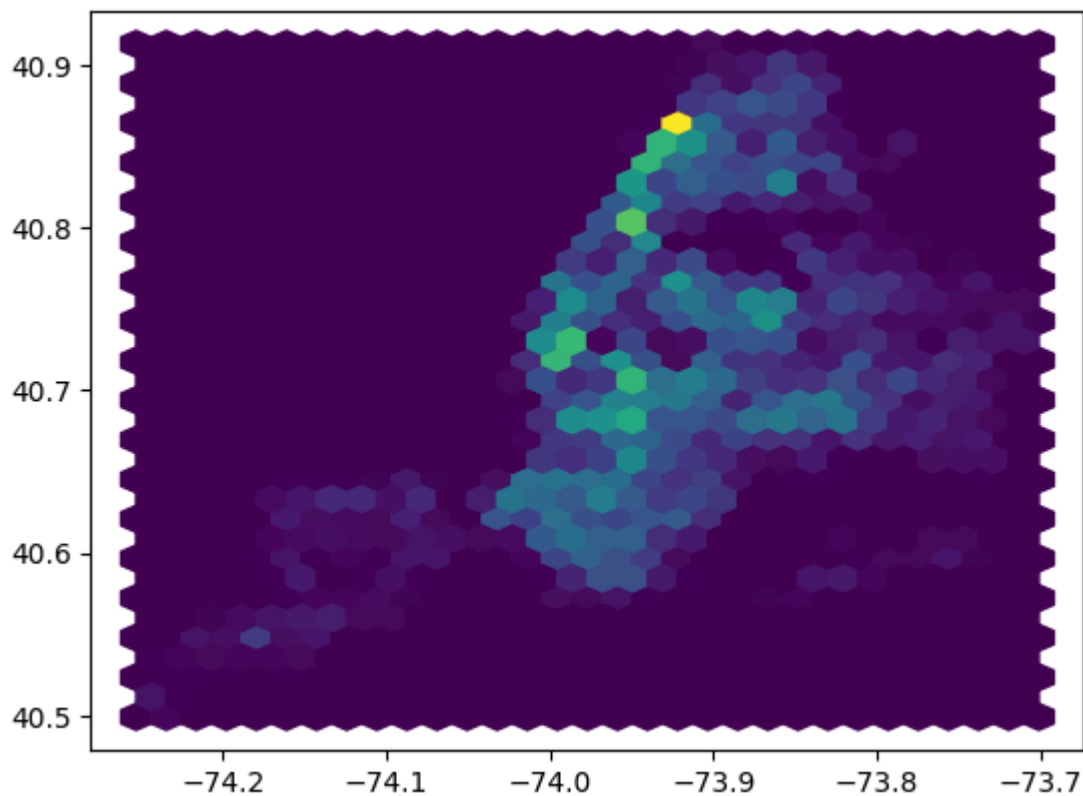


Create a scatter and hexbin plot of the concentration of complaints across Brooklyn

```
In [21]: plt.scatter(data['Longitude'], data['Latitude'], alpha=0.5)
plt.show()
```

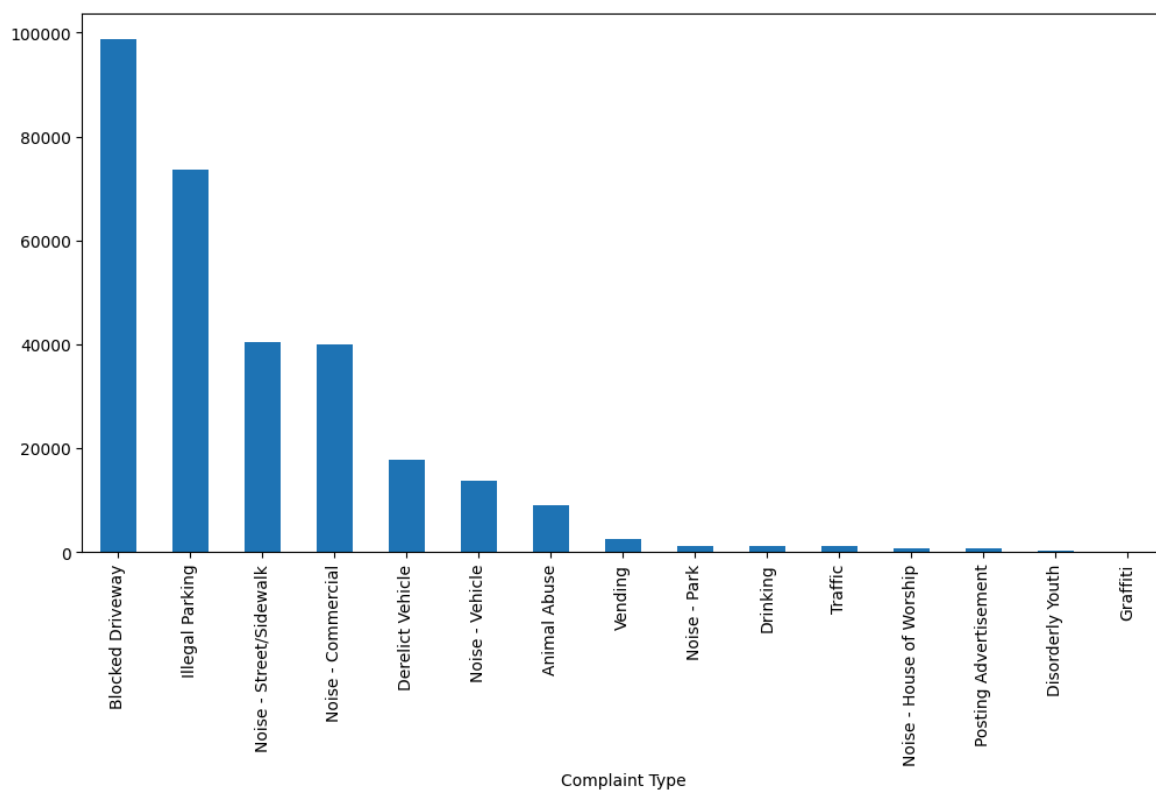


```
In [22]: plt.hexbin(data['Longitude'], data['Latitude'], gridsize=30)
plt.show()
```

Plot a bar graph to show the types of complaints

```
In [23]: b=data['Complaint Type'].value_counts()
b.plot(kind='bar', figsize=(12,6))
plt.show()
```



Check the frequency of various types of complaints for New York City

```
In [24]: nyc = data[data['City'] == 'NEW YORK']
print(nyc['Complaint Type'].value_counts())
```

```
Complaint Type
Noise - Street/Sidewalk    17544
Noise - Commercial        17195
Illegal Parking            11235
Noise - Vehicle            4142
Blocked Driveway          2525
Vending                   1551
Animal Abuse              1355
Derelict Vehicle           561
Noise - Park               425
Traffic                    331
Drinking                   267
Noise - House of Worship   162
Disorderly Youth           43
Posting Advertisement       40
Graffiti                   17
Name: count, dtype: int64
```

Find the top 10 complaint types

```
In [25]: top10 = data['Complaint Type'].value_counts().head(10)
top10
```

```
Out[25]: Complaint Type
Blocked Driveway    98668
Illegal Parking     73719
Noise - Street/Sidewalk 40444
Noise - Commercial  39919
Derelict Vehicle    17706
Noise - Vehicle     13763
Animal Abuse        8968
Vending             2451
Noise - Park        1289
Drinking            1124
Name: count, dtype: int64
```

Display the various types of complaints in each city

```
In [26]: city_complaints = data.groupby('City')['Complaint Type'].value_counts()
city_complaints
```

```
Out[26]: City      Complaint Type
ARVERNE  Blocked Driveway      49
          Illegal Parking      47
          Animal Abuse        45
          Derelict Vehicle     25
          Noise - Street/Sidewalk 18
          ...
Woodside  Illegal Parking     124
          Blocked Driveway     27
          Derelict Vehicle      8
          Noise - Street/Sidewalk 5
          Noise - Commercial    2
Name: count, Length: 585, dtype: int64
```

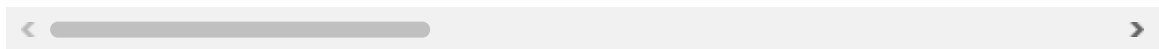
Create a DataFrame, df_new, which contains cities as columns and complaint types in rows

```
In [27]: df_new = pd.crosstab(data['Complaint Type'], data['City'])
df_new
```

Out[27]:

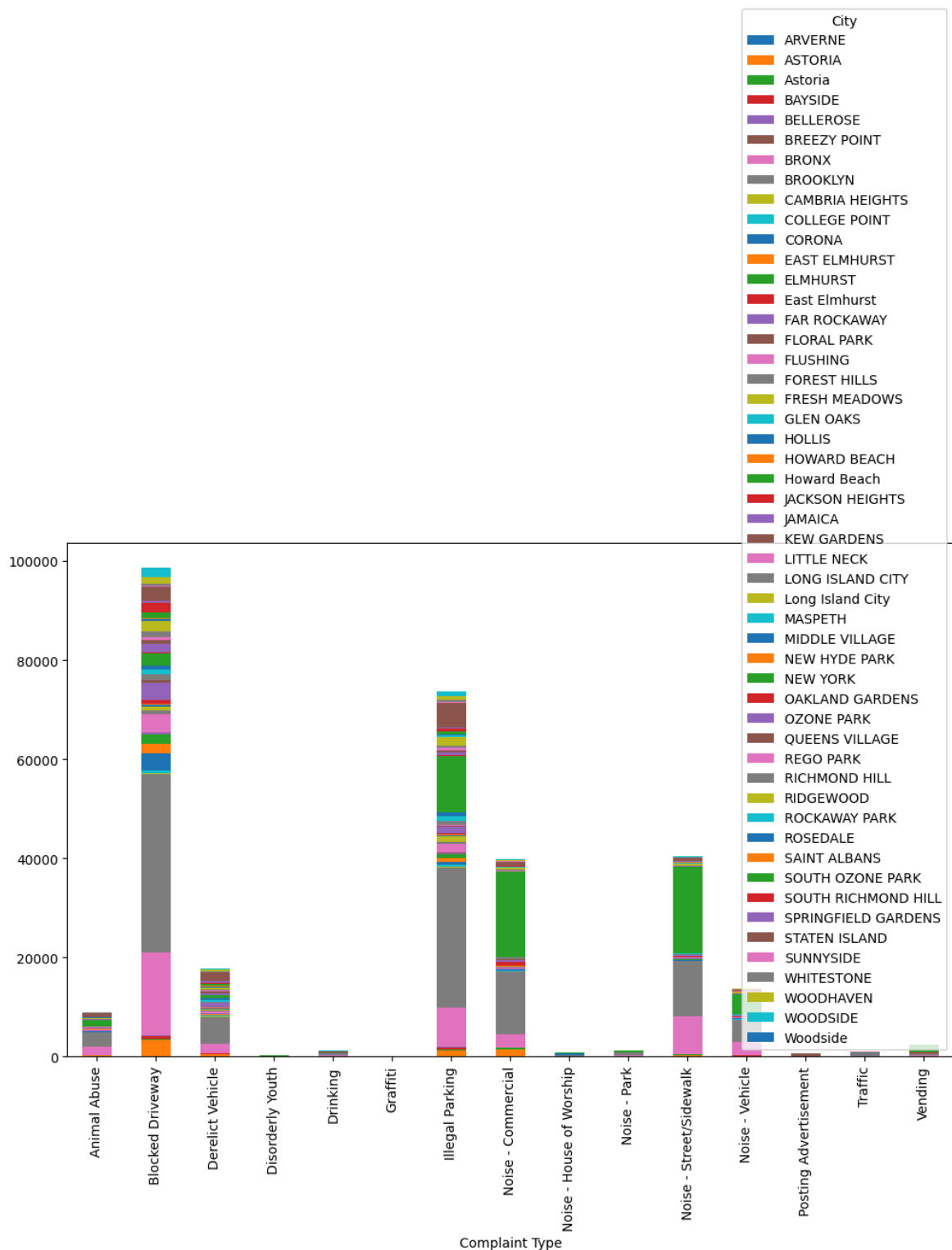
	City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX
Complaint Type								
Animal Abuse		45	149	0	48	13	2	1742
Blocked Driveway		49	3372	159	506	137	3	16789
Derelict Vehicle		25	345	14	181	82	3	1960
Disorderly Youth		1	4	0	1	2	0	50
Drinking		0	33	0	1	1	1	168
Graffiti		1	4	0	3	0	0	13
Illegal Parking		47	1096	277	453	98	14	7867
Noise - Commercial		2	1478	310	35	37	4	2611
Noise - House of Worship		13	17	0	2	1	0	61
Noise - Park		0	27	0	1	0	0	214
Noise - Street/Sidewalk		18	299	144	5	9	1	7584
Noise - Vehicle		6	148	0	16	7	1	2679
Posting Advertisement		0	2	0	0	0	0	13
Traffic		0	17	0	2	3	0	85
Vending		1	32	0	0	0	0	281

15 rows × 51 columns



Draw another chart that shows the types of complaints in each city in a single chart, where different colors show the different types of complaints

```
In [28]: df_new.plot(kind='bar', stacked=True, figsize=(12,7))
plt.show()
```



Sort the complaint types based on the average Request_Closing_Timegrouping them for different locations

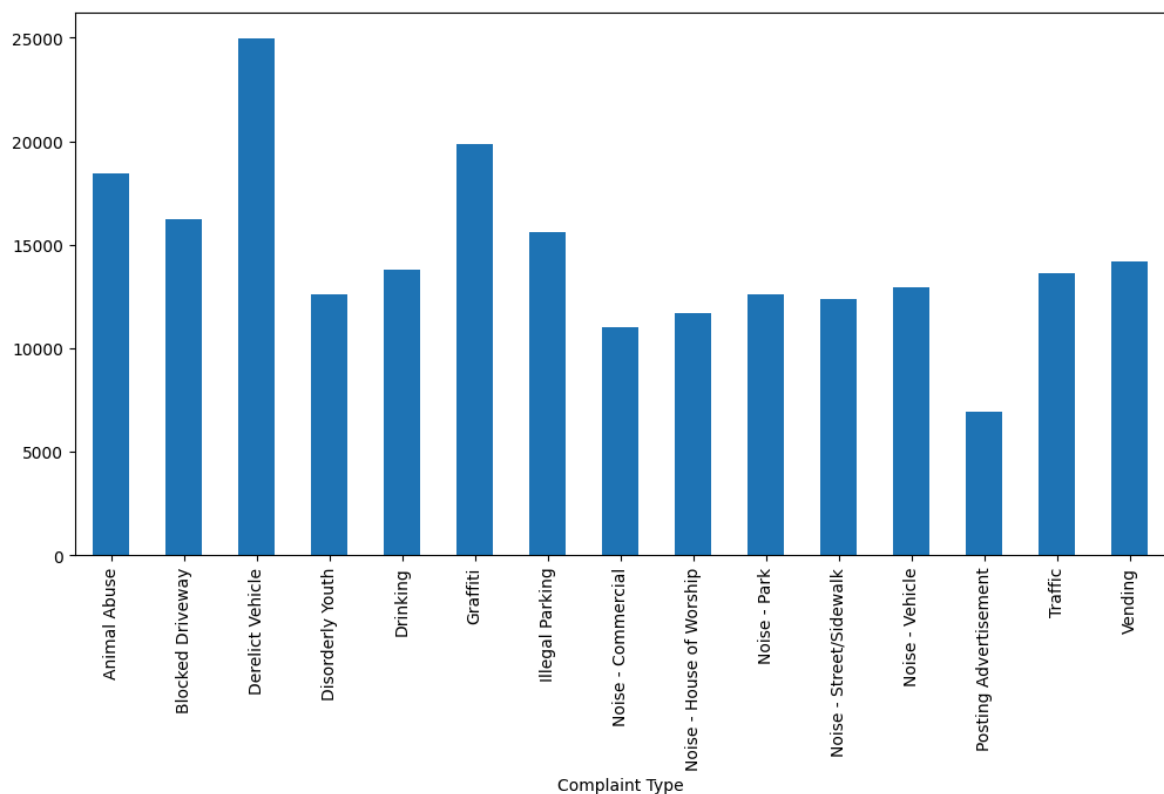
```
In [29]: avg_time = data.groupby(['City', 'Complaint Type'])['Time_in_seconds'].mean()
sorted_avg_time = avg_time.sort_values()
print(sorted_avg_time)
```

City	Complaint Type	
ELMHURST	Disorderly Youth	461.000000
RIDGEWOOD	Posting Advertisement	895.000000
ARVERNE	Vending	1735.000000
KEW GARDENS	Posting Advertisement	1872.000000
FRESH MEADOWS	Traffic	1901.500000
...		
GLEN OAKS	Derelect Vehicle	51497.673077
BELLEROSE	Derelect Vehicle	55772.987805
GLEN OAKS	Noise - Street/Sidewalk	62192.500000
CAMBRIA HEIGHTS	Traffic	64111.000000
MASPETH	Noise - Park	88120.000000

Name: Time_in_seconds, Length: 585, dtype: float64

Visualize the average of Request_Closing_Time

```
In [30]: avg_response = data.groupby('Complaint Type')['Time_in_seconds'].mean()
avg_response.plot(kind='bar', figsize=(12,6))
plt.show()
```



Identify the significant variables by performing statistical analysis using p-values

```
In [31]: from scipy.stats import f_oneway

groups = [group['Time_in_seconds'].dropna().values
           for name, group in data.groupby('Complaint Type')]

f_stat, p_value = f_oneway(*groups)

print("P-value:", p_value)
```

P-value: 0.0

Perform a Kruskal-WallisH test

```
In [32]: from scipy.stats import kruskal

# Create groups based on complaint type
groups = [
    group['Time_in_seconds'].dropna().values
    for name, group in data.groupby('Complaint Type')
]

# Apply test
h_stat, p_value = kruskal(*groups)

print("H-statistic:", h_stat)
print("P-value:", p_value)
```

H-statistic: 9593.727168223624

P-value: 0.0

Fail to reject H0: All sample distributions are equal

Reject H0: One or more sample distributions are not equal

```
In [33]: if p_value > 0.05:
          print("Fail to reject H0")
        elif p_value < 0.05:
          print("Reject H0")
```

Reject H0

Present your observations

“The analysis showed that complaint types and response times vary significantly across cities, and statistical testing confirmed that these differences are meaningful. This can help improve operational efficiency and decision-making.”